Absolute Return Forecasting: From Linear Models to LSTM

Edouard Blanc Clément Dardàre
Julien Simeoni

Université Paris Dauphine-PSL, Master 2 Machine Learning for Finance

March 29, 2026

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Objective . . . 3
1.3 Dataset . . . 3
1.4 Evaluation . . . 4
1.5 Paper Structure . . . 4

2 Data **4**
2.1 Dataset Construction . . . 4
2.2 Descriptive Statistics . . . 5
2.3 Log-Return Time Series . . . 5
2.4 Realized Volatility and Regime Identification . . . 6
2.5 Autocorrelation Analysis . . . 6
2.6 Feature Correlation Structure . . . 7
2.7 Summary . . . 7

3 Linear Regression Baseline **7**
3.1 Standard Linear Regression . . . 7
3.1.1 Setup . . . 7
3.1.2 Results . . . 8
3.1.3 Coefficient Analysis . . . 8
3.1.4 Predicted vs. Actual . . . 9
3.2 Feature Engineering . . . 9
3.3 Regime Analysis . . . 9
3.4 GARCH(1,1) Baseline . . . 10
3.4.1 Estimated Parameters . . . 10
3.4.2 Regime Performance . . . 10
3.5 Summary of Limitations . . . 11

4 Feedforward Neural Network **12**
4.1 Architecture 12
4.2 Training Setup 12
4.3 Results 12
4.4 Analysis 12
4.5 Limitations 12

5 Recurrent Neural Network **12**
5.1 Architecture 12
5.2 Training Setup 12
5.3 Results 12
5.4 Limitations 12

6 Long Short-Term Memory **12**
6.1 Architecture 12
6.2 Training Setup 12
6.3 Results 12
6.4 Cross-Model Comparison 12
6.5 Conclusion 12

1 Introduction

1.1 Motivation

Accurately forecasting the magnitude of future asset returns is a central problem in quantitative finance. While the *direction* of price changes is notoriously difficult to predict, the *amplitude* of those changes (often proxied by realized volatility) exhibits strong and exploitable serial dependence. This phenomenon is characterised by *volatility clustering*: large price moves tend to be followed by large moves, and calm periods persist until a new shock arrives.

Predicting the absolute return of the S&P 500 index therefore serves a dual purpose: it is both a well-posed statistical problem with a meaningful signal in the data and a practically relevant task for risk management, option pricing, and portfolio construction.

1.2 Objective

This project empirically compares a progression of models of increasing expressive power applied to the same forecasting task:

1. **Linear Regression** — a transparent but structurally constrained baseline.
2. **GARCH(1,1)** — the industry-standard parametric volatility model.
3. **Feedforward Neural Network (FNN)** — a non-linear model that treats the input as a flat, orderless feature vector.
4. **Recurrent Neural Network (RNN)** — a sequential model that can exploit the temporal ordering of lags.
5. **Long Short-Term Memory (LSTM)** — a gated architecture designed to capture long-range dependencies.

Rather than searching for the best possible model, the goal is to *understand the limitations of each architecture* and to show how each successive model overcomes (some of) the shortcomings of its predecessor.

1.3 Dataset

The dataset is built from the daily log-returns of the SPY ETF (S&P 500 tracker), downloaded via the `yfinance` library. The sample spans **February 1993 to March 2026**, yielding **8 343 daily observations**.

The forecasting problem is formulated as a sliding-window regression:

- **Input** X_t: the L most recent daily log-returns $(r_{t-1}, r_{t-2}, \ldots, r_{t-L})$, with $L \geq 10$.
- **Target** y_t: the realized absolute return on day t, computed as the rolling standard deviation of returns over a short window (a proxy for next-day volatility).

1.4 Evaluation

Models are evaluated using three metrics, computed on a held-out test set:

- **MAE** (Mean Absolute Error) — primary metric for cross-model comparison.
- **RMSE** (Root Mean Squared Error) — penalises large errors more heavily.
- **QLIKE** (Quasi-Likelihood loss) — a standard volatility-forecasting loss that penalises systematic under-prediction.

Because market conditions vary dramatically, all models are also evaluated separately on two **market regimes** identified from the data:

- **Calm regime** (≈90% of observations): low, stable volatility.
- **Crisis regime** (≈10% of observations): sharp spikes corresponding to the dot-com crash, the 2008 Global Financial Crisis, and the COVID shock.

The cross-model summary (Section 6) reports MAE only.

1.5 Paper Structure

Section 2 describes the dataset and its key statistical properties. Section 3 presents the linear regression and GARCH baselines. Sections 4–6 progressively introduce the neural network architectures. A final comparison table closes the report.

2 Data

2.1 Dataset Construction

Daily log-returns for the SPY ETF are downloaded via `yfinance` and stored locally in `data/df_sp_500_log_ret.csv`. The raw return on day t is defined as

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right),$$

where P_t denotes the adjusted closing price. A sliding-window dataset is then constructed with a look-back of $L \geq 10$ lags:

$$\begin{aligned} X_t &= (r_{t-1},\, r_{t-2},\, \ldots,\, r_{t-L}) \in \mathbb{R}^L, \\ y_t &= \sigma_t \in \mathbb{R}_+, \end{aligned}$$

where σ_t is the realized absolute return (rolling standard deviation of returns over a short window), serving as a proxy for next-day volatility.

2.2 Descriptive Statistics

Table 1 reports summary statistics for the full sample.

Table 1: Descriptive statistics for S&P 500 daily log-returns (SPY, 1993–2026).

Statistic	Value
Observations	8 343
Start date	1993-02-01
End date	2026-03-24
Mean	0.0004
Std deviation	0.0117
Minimum	−0.1159
Maximum	+0.1356
Skewness	−0.248
Kurtosis	11.42

The distribution exhibits **negative skewness** (large negative shocks more common than large positive ones) and **excess kurtosis** of 11.42, confirming heavy tails relative to a Gaussian distribution. Both features are consistent with the well-known stylised facts of equity returns.

2.3 Log-Return Time Series

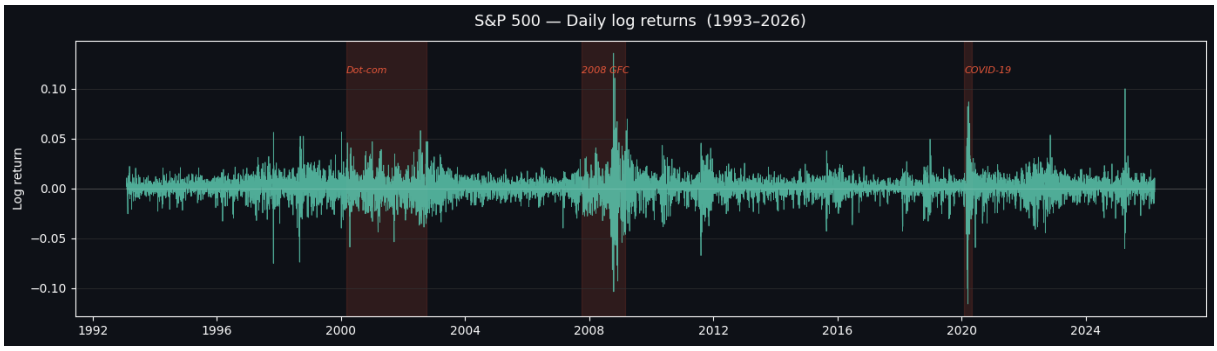

Figure 1: S&P 500 daily log-returns. Three major stress periods are clearly visible.

Returns oscillate around zero with no discernible trend, consistent with the efficient market hypothesis. Three major stress episodes are visible: the dot-com crash (2000–2002), the Global Financial Crisis (2008–2009), and the COVID shock (2020). These episodes exhibit the characteristic feature of **volatility clustering**: large moves arrive in bursts and calm periods are interspersed.

2.4 Realized Volatility and Regime Identification

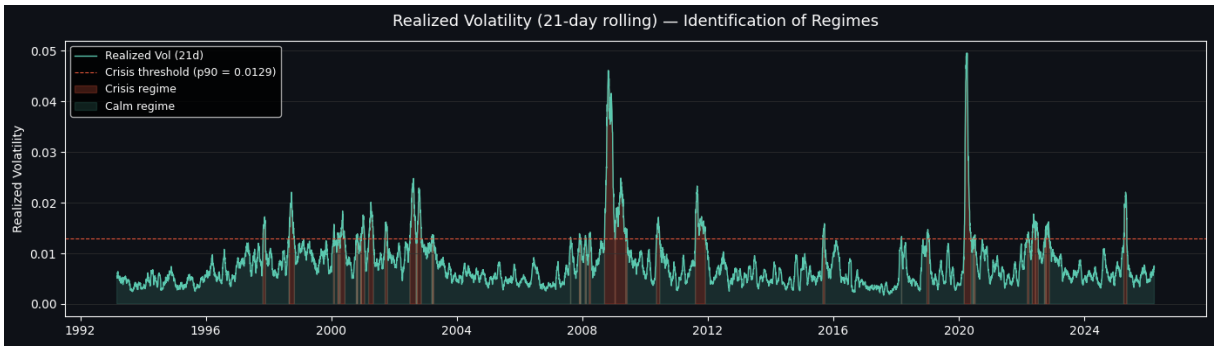

Figure 2: Realized absolute return (target variable y_t). The dashed line indicates the 90th-percentile threshold used to define the crisis regime.

A threshold at the **90th percentile** ($\sigma^* = 1.117$) is used to split the sample into two regimes:

Table 2: Market regime statistics.

Regime	Share of observations	Characterization
Calm	90%	Low, stable volatility
Crisis	10%	Sharp spikes; three major episodes

The pronounced asymmetry in regime frequency is critical: a model that performs well on average may still fail badly during the 10% of days that matter most for risk management.

2.5 Autocorrelation Analysis

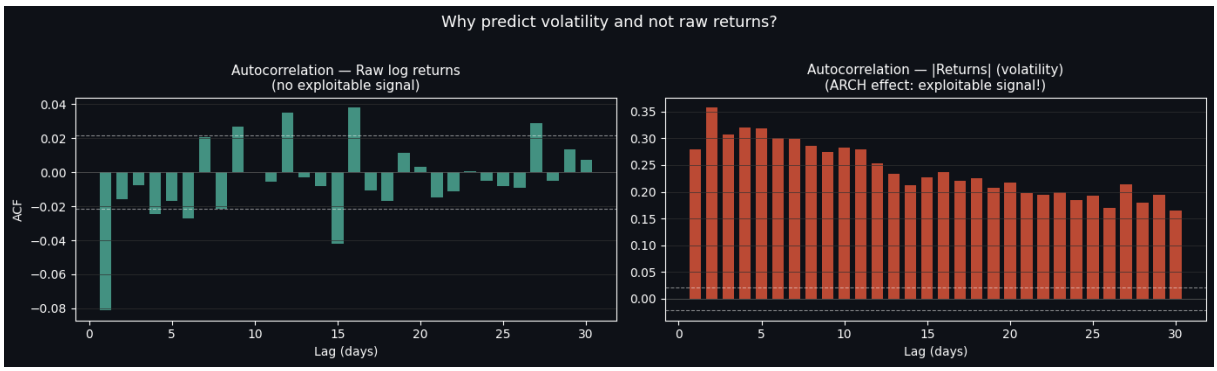

Figure 3: Autocorrelation of raw log-returns (near zero) vs absolute log-returns (strongly positive). The latter justifies using past absolute returns as features.

Two key findings:

- **Raw log-returns**: autocorrelation is essentially zero at all lags — the return series itself is unpredictable.

- **Absolute log-returns / realized volatility**: autocorrelation is strongly and persistently positive (the ARCH effect), justifying the use of past lagged returns as features for predicting future volatility.

2.6 Feature Correlation Structure

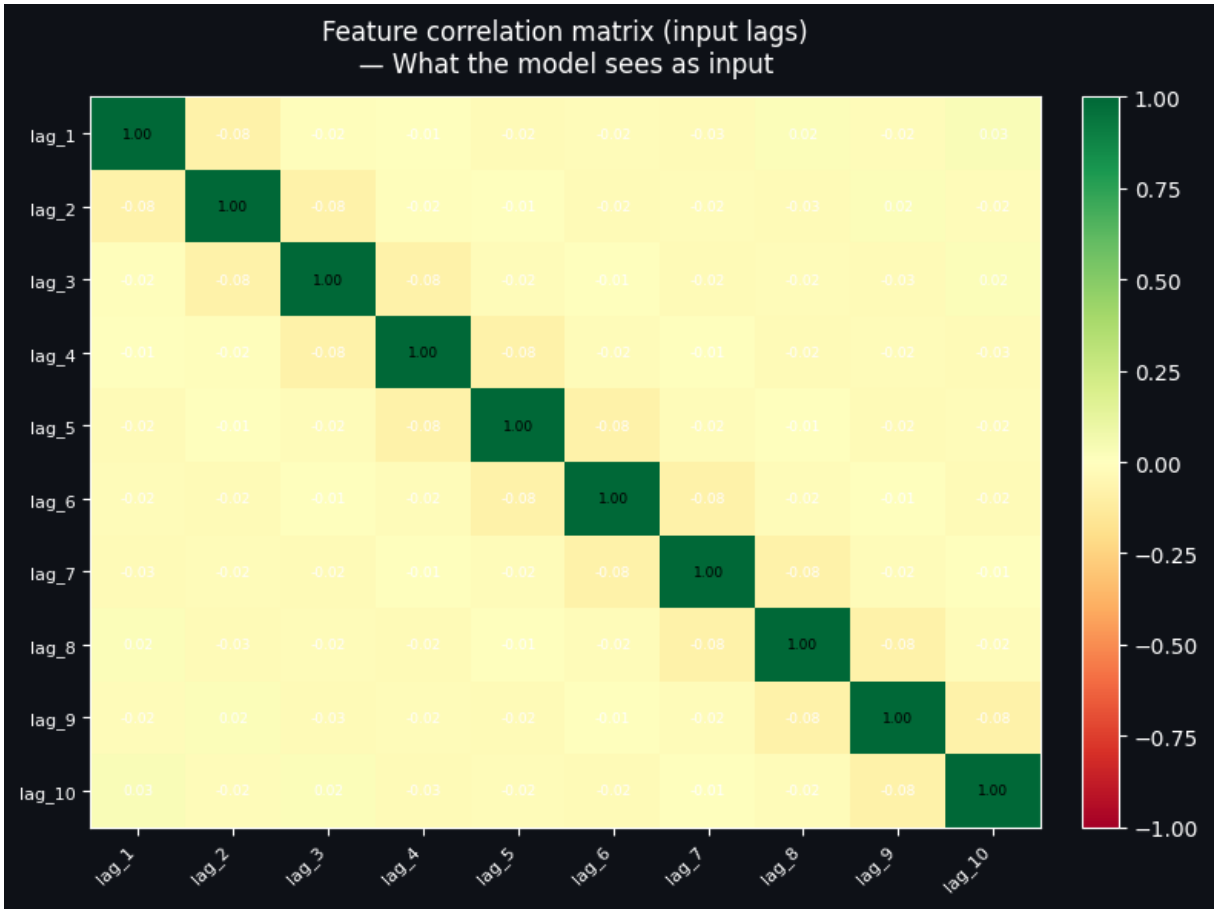

Figure 4: Pearson correlation matrix of the 10 lag features. Adjacent lags are positively correlated, reflecting the persistence of volatility regimes.

Adjacent lags are positively correlated due to volatility clustering: a large move on day $t-1$ is likely to be followed by a large move on day $t-2$. This introduces mild multicollinearity for linear models and gives sequential models (RNN, LSTM) a structural advantage.

2.7 Summary

Three statistical properties of the data guide the modelling choices throughout this report:

1. **Raw returns are unpredictable** (near-zero ACF) — the target must be the magnitude, not the sign, of returns.
2. **Volatility is autocorrelated** (ARCH effect) — past lags carry genuine predictive information.
3. **Two regimes coexist** — calm and crisis periods have fundamentally different statistical properties; a good model must handle both.

3 Linear Regression Baseline

This section establishes two classical baselines: ordinary least squares (OLS) regression and GARCH(1,1). Both are transparent and well-understood, making their failure modes informative for motivating the neural network architectures that follow.

3.1 Standard Linear Regression

3.1.1 Setup

An OLS model is trained on the 10 signed lag features with a standard train/test split. Features and target are scaled with a `StandardScaler` fitted on the training set only. The model takes the form

$$\hat{y}_t = \beta_0 + \sum_{k=1}^{10} \beta_k \, r_{t-k}.$$

3.1.2 Results

Table 3: Linear regression performance on the test set (scaled units).

Model	MAE	RMSE	QLIKE
Linear Regression (raw lags)	0.638	0.933	16.79
Linear Regression (engineered)	0.598	0.874	7.46

3.1.3 Coefficient Analysis

Table 4: OLS regression coefficients ranked by absolute magnitude.

Feature	Coefficient	$\lvert\beta_k\rvert$ rank
lag_9	-0.180	1
lag_8	-0.164	2
lag_10	-0.159	3
lag_7	-0.158	4
lag_5	-0.123	5
lag_6	-0.110	6
lag_3	-0.106	7
lag_1	-0.094	8
lag_4	-0.084	9
lag_2	-0.060	10

All coefficients are *negative* and small. This reveals the first fundamental limitation:

> **Symmetry blindness.** Linear regression on signed returns cannot treat $+x$ and $-x$ identically. Predicting volatility requires $f(x) \approx f(-x)$ (the absolute value function), which a linear model cannot represent. The model "gives up" by assigning small negative coefficients and predicting near-zero values for all inputs.

3.1.4 Predicted vs. Actual

[Figure placeholder]
Linear regression predictions (orange) vs actual realized volatility (blue). The model predicts near-zero at all times; largest residuals occur during crises.

Figure 5: Linear regression: predicted vs. actual realized absolute return. The model systematically underpredicts during crisis episodes.

The model produces a near-constant prediction close to zero throughout the entire sample. Residuals are always positive and are dramatically amplified during crisis episodes.

3.2 Feature Engineering

Squaring the lag features forces the symmetry the model cannot learn from signed returns: $r_{t-k}^2 = |r_{t-k}|^2$ treats positive and negative shocks of equal magnitude identically. A *recency ratio* feature $r_{t-1}/\text{mean}(r_{t-2:t-5})$ is also added to capture short-term momentum in volatility.

Table 5: Impact of feature engineering on linear regression performance.

Features	MAE	RMSE	QLIKE
Raw signed lags	0.638	0.933	16.79
Squared lags + recency ratio	0.598	0.874	7.46
Improvement (Δ)	-6.3%	-6.4%	-55.6%

The QLIKE loss drops by 56%, showing that manual feature engineering substantially reduces systematic under-prediction. This foreshadows what the FNN can learn *automatically* from raw lags.

3.3 Regime Analysis

Table 6: Linear regression performance by market regime (raw lag features).

Regime	MAE	RMSE	QLIKE
Full	0.638	0.933	16.79
Calm	0.488	0.587	15.49
Crisis	1.987	2.367	28.46
Crisis / Calm ratio	4.1×	4.0×	1.8×

During crisis periods, MAE is **4× higher** than during calm periods. The model systematically underpredicts every volatility spike because it has no mechanism to detect a regime shift. This is the second structural limitation:

> **Regime blindness.** A single linear model fitted on the full sample cannot adapt its behaviour to different market conditions.

3.4 GARCH(1,1) Baseline

The GARCH(1,1) model [?] provides a parametric volatility benchmark. The conditional variance follows:

$$\sigma_t^2 = \omega + \alpha\,\epsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2,$$

where $\epsilon_t = r_t/\sigma_t$ are standardised residuals.

3.4.1 Estimated Parameters

Table 7: GARCH(1,1) maximum-likelihood parameter estimates.

Parameter	Estimate	Interpretation
ω (baseline variance)	0.0159	Long-run variance floor
α (shock sensitivity)	0.094	Reaction to new shocks
β (persistence)	0.895	Memory of past variance
$\alpha + \beta$ (total persistence)	0.989	Near unit-root variance process
Half-life of shock decay	60 days	Volatility shocks dissipate slowly

The near-unit-root persistence ($\alpha + \beta = 0.989$) confirms that volatility shocks take approximately **60 days** to decay to half their peak level.

3.4.2 Regime Performance

Table 8: GARCH(1,1) performance by market regime (unscaled returns).

Regime	MAE	RMSE	QLIKE
Full	0.00583	$0.000069^{\dagger}$	−3.98
Calm	0.00506	$0.000043^{\dagger}$	−4.17
Crisis	0.01281	$0.000296^{\dagger}$	−2.27
Crisis / Calm ratio	2.5×	6.9×	—

† These are MSE values; RMSE= $\sqrt{\text{MSE}}$.

GARCH improves on linear regression during calm periods but exhibits the same pattern of regime degradation: crisis MAE is 2.5× higher than calm MAE. Despite its elegant theoretical motivation, GARCH suffers from the third structural limitation:

> **Fixed functional form.** GARCH assumes a specific parametric update rule for variance. It cannot detect structural breaks or non-linear regime transitions.

3.5 Summary of Limitations

Table 9: Comparison of linear baseline models (MAE, full sample).

Model	MAE (full)	MAE (crisis)
Linear Regression (raw lags)	0.638	1.987
Linear Regression (engineered)	0.598	—
GARCH(1,1)	0.00583	0.01281

Three structural limitations motivate the turn to neural networks:

1. **Symmetry blindness** (linear regression on signed inputs cannot represent $|x|$).
2. **Regime blindness** (no model adapts its behaviour during crises).
3. **Fixed functional form** (GARCH assumes a pre-specified update rule).

A feedforward neural network with non-linear activations directly addresses limitation 1, and deeper architectures are expected to partially address limitations 2 and 3.

4 Feedforward Neural Network

4.1 Architecture

4.2 Training Setup

4.3 Results

4.4 Analysis

4.5 Limitations

5 Recurrent Neural Network

5.1 Architecture

5.2 Training Setup

5.3 Results

5.4 Limitations

6 Long Short-Term Memory

6.1 Architecture

6.2 Training Setup

6.3 Results

6.4 Cross-Model Comparison

6.5 Conclusion